

Lecture 8 Relational Database Design Purpose & First Normal Form

United International College

Outline

- **Features of Good Relational Design**
- Atomic Domains and First Normal Form
- Functional Dependency Theory
- BCNF
- 3rd Normal Form
- Multivalued Dependencies and Decomposition
- Database-Design Process

What is a good Design?

Usually, a good design allows us to

- retrieve information easily,
- obtain all constraints,
- minimize redundancies, and
- have no ambiguous.

This is accomplished systematically by using *normal forms*.

The UIC database Schema

- $program = (\underline{p_code}, p_name, division, director)$
- $course = (\underline{c_name}, credits, domain, c_number)$
- $offer = (\underline{p_code}, \underline{c_name})$
- $section = (\underline{c_name}, \underline{s_number}, \underline{sem}, venue, time, instructor)$
- $student = (\underline{id}, s_name, year, gpa, major)$
- $instructor = (\underline{id}, i_name, title, salary, program)$
- $contact = (\underline{id}, \underline{phone})$
- $book = (\underline{ISBN}, b_title, author)$
- $borrow = (\underline{id}, \underline{ISBN}, return, borrow)$

This logical design is good enough because the ER diagram is carefully designed. But sometimes, the life is not this easy. Some schemas may be too large, while some others may be too small. Problems will arise.

Larger Schemas?

Suppose that instead of the schemas

$course = (\underline{c_name}, credits, domain, c_number)$

$offer = (\underline{p_code}, c_name)$

we have a larger schema

$course_info = (c_name, credits, domain, c_number, p_code)$

Note: p_code is the code of the program which offers the course.

- What is the key of this schema?
- Why this is not a good design?

Larger Schemas?

course_info = (*c_name*, *credits*, *domain*, *c_number*, *p_code*)

- What is the key of this schema?
 - *c_name* itself cannot be a key. When a course is offered by multiple programs, *c_name* can no longer identify tuples.

<i>c_name</i>	<i>credits</i>	<i>domain</i>	<i>c_number</i>	<i>p_code</i>
:	:	:	:	:
Database	3	COMP	3013	CST
Database	3	COMP	3013	DS
:	:	:	:	:

- The key has to be *c_name*, *p_code* (two attributes).

Larger Schemas?

course_info = (*c_name*, *credits*, *domain*, *c_number*, *p_code*)

- Why this is not a good design?
 - From the example, we see redundancy occurs.

<i>c_name</i>	<i>credits</i>	<i>domain</i>	<i>c_number</i>	<i>p_code</i>
⋮	⋮	⋮	⋮	⋮
Database	3	COMP	3013	CST
Database	3	COMP	3013	DS
⋮	⋮	⋮	⋮	⋮

- The reason is that *credits*, *domain*, *c_number* are decided by the course, but not by the program which offers the course.

What About Smaller Schemas?

- Suppose we start with *course_info*. How do we know to split (**decompose**) it into *course* and *offer*?
- In fact, decompositions cannot be naïvely applied.
- Sometimes decompositions are useless.
- For example,

program = (*p_code*, *p_name*, *division*, *director*)

do not need to be decomposed into

program = (*p_code*, *p_name*, *division*)

PD = (*p_code*, *director*)

- The original schema does not create redundancies.
- After decomposition, you need a “JOIN” to find the director for a program if you only know the program name, which is time consuming.

What About Smaller Schemas?

- Sometimes naïve decompositions create inconsistency.
- For example,

$student = (\underline{id}, s_name, year, gpa, major)$

is decomposed into (pay attention to the keys)

$stu = (\underline{id}, s_name)$

$stuinfo = (\underline{s_name, year, gpa, major})$

<u>id</u>	s_name	year	gpa	major
⋮	⋮	⋮	⋮	
1001000654	Dave	5	2.01	TESL
1701026104	Dave	1	3.41	CST
⋮	⋮	⋮	⋮	

<u>id</u>	s_name
⋮	⋮
1001000654	Dave
1701026104	Dave
⋮	⋮

<u>s_name</u>	<u>year</u>	<u>gpa</u>	<u>major</u>
⋮	⋮	⋮	
Dave	5	2.01	TESL
Dave	1	3.41	CST
⋮	⋮	⋮	

What About Smaller Schemas?

- Suppose we want to find the major of the student by IDs, we have to join *stu* and *stu_info*. This creates some fake data.

<u>id</u>	<u>s_name</u>	JOIN	<u>s_name</u>	<u>year</u>	<u>gpa</u>	<u>major</u>
⋮	⋮		⋮	⋮	⋮	
1001000654	Dave		Dave	5	2.01	TESL
1701026104	Dave		Dave	1	3.41	CST
⋮	⋮		⋮	⋮	⋮	

<u>id</u>	<u>s_name</u>	<u>year</u>	<u>gpa</u>	<u>major</u>
⋮	⋮	⋮	⋮	
1001000654	Dave	5	2.01	TESL
1001000654	Dave	5	2.01	CST
1701026104	Dave	1	3.41	TESL
1701026104	Dave	1	3.41	CST
⋮	⋮	⋮	⋮	

What About Smaller Schemas?

- Which schema should be decomposed, and which should not?
- The reason for decomposing *course_info* is already given.
- The value of *c_name* can distinguish the values of *credits*, *domain*, and *c_number*.
- This distinguishment is denoted as a **functional dependency** (will be on this later):

$$c_name \rightarrow credits, domain, c_number$$

- In *course_info*, *c_name* is not a candidate key. Thus, the values of *credits*, *domain*, and *c_number* may have repetition. This indicates the need to decompose *course_info*.
- This property does not hold in the last two examples.

Outline

- Features of Good Relational Design
- **Atomic Domains and First Normal Form**
- Functional Dependency Theory
- BCNF
- 3rd Normal Form
- Multivalued Dependencies and Decomposition
- Database-Design Process

First Normal Form

- A domain is **atomic** if its elements are considered to be indivisible units.
 - Examples of non-atomic domains:
 - Set of names, composite attributes
 - Identification numbers like CS101, that can be broken up into parts.
- A relational schema R is in **first normal form** if the domains of all attributes of R are atomic.
- Non-atomic values complicate storage and encourage redundant (repeated) storage of data.

First Normal Form

- Atomicity is actually a property of how the elements of the domain are used.
- Example: Strings would normally be considered indivisible.
- Suppose that students are given roll numbers which are strings of the form *CS0012* or *EE1127*.
- If the first two characters are extracted to find the department, the domain of roll numbers is not atomic.
- Doing so is a bad idea: leads to the encoding of information in application program rather than in the database.
- Database users have to know the encoding before developing applications.

First Normal Form

- Here is another example of the violation of the first normal form. This is very common to new database designers.
- Suppose we want to record the favorite books of each student.
- This design is not good.

<u><i>id</i></u>	<i>s_name</i>	<i>favorite_books</i>
1001000654	Dave	Three body, MacBeth, Database
1701026104	Steven	Three Body
⋮	⋮	⋮

- The attribute *favorite_books* is not atomic.
- A better design can be

<u><i>id</i></u>	<i>s_name</i>	<u><i>favorite_book</i></u>
1001000654	Dave	Three body
1001000654	Dave	MacBeth
1001000654	Dave	Database
1701026104	Steven	Three Body
⋮	⋮	⋮

End of Lecture 8